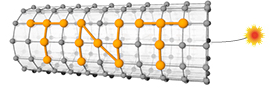


Tensor Network Theory



Part 1: Overview of core library and tensor operations

Sarah Al-Assam¹, Stephen R. Clark² and Dieter Jaksch¹

¹Department of Physics, University of Oxford, ²Department of Physics, University of Bath

June 7, 2022

In this technical paper we introduce the Tensor Network Theory (TNT) library – an open-source software project aimed at developing robust, easy to use and highly optimised code for TNT calculations, available at <http://www.tensornetworktheory.org>. The aims of this paper are threefold: to (i) give a brief but broad introduction to TNT, (ii) give an overview of the structure of TNT library, and (iii) describe in detail the core tensor features of the library.

1 Introduction

Tensor Network Theory (TNT) is a powerful numerical approach for efficiently storing and processing high dimensional complex multi-linear data. It involves rather general concepts applicable to physics [1, 2, 3, 4], mathematics [5] and computer science [6]. In the context of physics, which will be our focus here, the data in question involves formulating descriptions of many-body quantum states $|\psi\rangle$ and the linear operators which act upon them. In this introductory section we will give a brief overview of the quantum many-body problem, outlining why it is important and why solving it presents such an enormous undertaking. Then we will outline the basic foundations of TNT that is squarely aimed at solving these problems for highly relevant cases.

1.1 The many-body problem

Understanding the collective behaviour of interacting many-body quantum systems remains an outstanding challenge in modern physics. Indeed strong correlation between electrons gives rise to a rich range of phenomenon, such as superconductivity, antiferromagnetic ordering, and topological spin liquids, which are both intriguing and functionally

relevant [7]. Further to this in the past decade interest in the coherent many-body dynamics of these systems far-from-equilibrium has dramatically intensified. On the one hand this activity has been propelled by developments in cold-atom experiments [8]. These allow for the realisation of controllable, well isolated strongly interacting many-body systems whose evolution can be tracked in real time. On the other hand recent advances in ultrafast THz laser science has now opened up new vistas of experiments probing and controlling solid-state systems [9]. In particular, selective large amplitude laser excitation of collective modes of a solid may allow for ultrafast switching between different broken-symmetry phases. This not only includes melting equilibrium long-ranged order, such as charge-density waves and superconductivity, but even more remarkably inducing such order with light in regimes where none existed in equilibrium.

A cornerstone of theoretical studies into these systems is the concept of minimal lattice models believed to capture the key physics, e.g. Hubbard-like Hamiltonians defined for one or more relevant electronic bands [10]. Given such a model understanding phenomenon seen in experiments ideally involves solving two key problems: (A) finding the ground state or low-lying excited states of the system, and (B) time-evolving the system from a given initial state according to some quench or periodic driving of the model. Despite their simplicity minimal models are extremely difficult to study; most of them cannot be solved exactly, the regimes of interest cannot be handled perturbatively, and widespread correlations make mean-field methods conceptually inadequate.

Numerical methods attacking the full quantum many-body problem are therefore essential. However, a fundamental roadblock is encountered – for a system composed of N separate d dimensional degrees of freedom (sites), its quantum state possesses

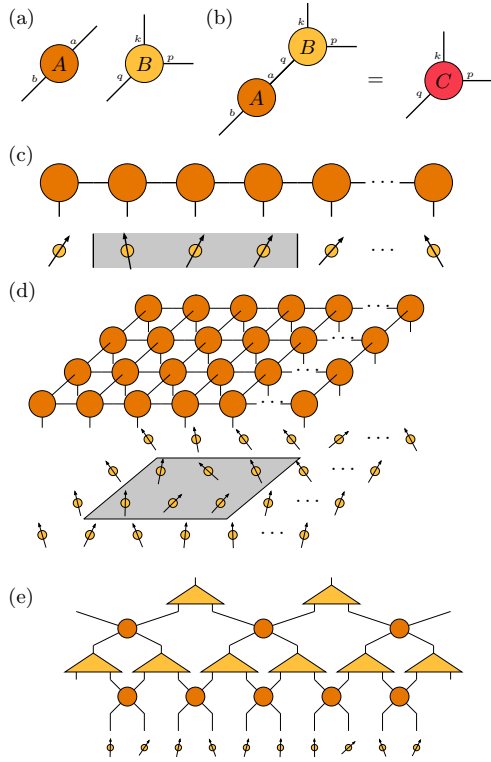


Figure 1: (a) A graphical depiction of an order-2 tensor A and an order-3 tensor B . (b) The contraction of tensors A and B to form a new order-3 tensor C shown here graphically as the joining of the corresponding legs. (c) An MPS tensor network for 1D systems. The vertical legs are open (uncontracted) and correspond to the physical states of sites of a 1D lattice shown directly beneath. (d) A PEPS tensor network for 2D systems. Again vertical legs map to sites of an underlying lattice. In both (c) and (d) a region of the system is shaded and its boundary with the rest of the system is outlined by the thick lines. (e) A MERA tensor network for 1D systems.

d^N complex amplitudes $\psi_{j_1 j_2 \dots j_N} = \psi_{\mathbf{j}}$ defined by N physical indices $\mathbf{j} = j_1, j_2, \dots, j_N$. This exponential scaling of the Hilbert space is aptly named the “curse of dimensionality” and limits exact diagonalisation to very small N . Furthermore numerous quantum Monte Carlo methods (e.g. based on imaginary-time projection), which otherwise successfully sidestep this issue for bosons, suffer from the notorious sign problem for crucial fermionic and frustrated lattice systems. Consequently, there exists an acute capability gap between the exciting experiments on strongly correlated systems and the theory trying to unravel their behaviour.

Over the past 15 years a new approach to the many-body problem has emerged called Tensor Network Theory (TNT) [1, 2, 3, 4]. So far it has pro-

vided powerful algorithms for solving problems (A) and (B) for 1D and 2D quantum lattice systems by exploiting insights from quantum information science. Central to this is entanglement entropy - a quantity that embodies most general the notion of quantum correlations between two subsystems. Remarkably studies have found that for gapped Hamiltonians with short-ranged interactions the entanglement between two regions in the system’s ground state, as well as low-lying excitations, is localized on their shared boundary [11]. As a result entanglement of a region scales as a so-called “area-law”, which is a striking contrast with a volume scaling commonly expected for an extensive quantity. This property constrains physical states of minimal models to occupy a very small “corner” of Hilbert space - a fact that provides a sought-after antidote to the curse of dimensionality. The central aim of TNT is to provide a highly flexible and unifying framework for constructing families of quantum states in this corner. It does so by explicitly designing them to exhibit a given entanglement structure, e.g. like satisfying the area law.

1.2 Tensor network theory

The building blocks of TNT are tensors, which in this context are essentially multidimensional arrays of complex numbers. It is useful to visualise them with a diagrammatic formalism where tensors are shapes with legs, each leg associated to a tensor index. In Fig. 1(a) an order-two tensor A_{ab} , equivalent to a matrix, is shown along with an order-three tensor B_{pqk} . If the indices a and q for the tensors A and B have the same dimension then they can be *contracted* to give a new order-three tensor $C_{pbk} = \sum_c A_{cb} B_{pck}$ shown in Fig. 1(b). Contraction is computationally equivalent to standard matrix multiplication and is diagrammatically represented by joining the corresponding legs of the tensors together. A many-body quantum state $|\psi\rangle$ with complex amplitudes $\psi_{\mathbf{j}}$ is therefore an order- N tensor - an intractably large structureless monolithic object.

To overcome this TNT attempts to factorise $\psi_{\mathbf{j}}$ into a network of low order tensors. Specifically we have a network G of vertices ν each with a tensor $T^{(\nu)}$. These tensor then possess a set of internal indices, of dimension at most χ , and may additionally possess physical indices j_ν . The edges of the network G describe how the internal legs of each tensor are to be joined together and therefore contracted. Thus, in this representation the order- N tensor of amplitudes for a quantum state emerge as the open

physical indices left over after performing all the contractions of the internal indices specified by the network G , denoted by the tensor trace $\text{tTr}[\dots]$ as

$$\psi_{\mathbf{j}} = \text{tTr}[\otimes_{\nu \in G} T^{(\nu)}]. \quad (1)$$

Important examples are given in Fig. 1(c)-(e) and are motivated by renormalisation concepts and dimensionality. The matrix product state (MPS) network in Fig. 1(c) is composed of a chain of tensors contracted together following the underlying structure of the lattice. The thicker open vertical legs are physical indices corresponding to lattice sites of the system shown below the network. Similarly in Fig. 1(d) the equivalent projected entangled pair state (PEPS) tensor network for a 2D square lattice is shown which generalises MPS. In Fig. 1(e) the multiscale entanglement renormalisation ansatz (MERA) is shown which possesses a hierarchically layered network where only the tensors in the bottom layer have physical indices.

The contraction of internal indices between tensors is directly related to the entanglement between the parts of the physical system the tensors are associated with. The more entanglement there is the larger the dimension χ has to be. If χ is allowed to scale exponentially with N then in principle a tensor network can describe any state $|\psi\rangle$ with volume scaling entanglement, but suffers the curse of dimensionality again. However, if χ is bounded the tensor network will only contain polynomially many elements, yet its network geometry can still allow for area-law scaling entanglement within the encoded state. This is illustrated in Fig. 1(c)-(d) where a patch of the physical system is shaded. The area-law states that the entanglement of this patch with the rest of the system scales with its boundary outlined by the thick lines. For 1D this is a constant, while in 2D it grows as the perimeter of the patch. For 1D MPS respect the area-law, but do not if they are applied to 2D by e.g. snaking a chain across a lattice [1]. In contrast the higher connectivity of PEPS ensures they continue to obey the area-law in 2D [2]. The MERA network in Fig. 1(e) can capture entanglement stratified over many length scales as seen for critical systems that logarithmically violate the area-law [4].

To solve problem (A) TNT algorithms employ a variational approach on the class of states described by a given tensor network with limited χ . Several properties set this variational approach apart from others. First, tensor networks provide an enormous class of variational states defined by many hundreds of thousands of parameters, depending on the value of χ . As such increasing χ can easily re-

fine the ansatz. Second, the bias of tensor networks is extremely general in the sense that it relates to entanglement, as opposed to any specific type of correlation or ordering. Indeed, beyond common spin or charging ordering tensor networks can also readily capture global topological properties and hidden string ordering via local symmetries of the tensors. Moreover this bias weakens quickly with increasing χ . But even more crucially this bias in the entanglement structure is commensurate to that of the corner of Hilbert space occupied by physically relevant states. As such tensor networks are expected to provide highly accurate and enormously compressed representations of physically relevant states and can be considered a quasi-exact variational optimisation.

To solve (B) a tensor network is used to capture dynamics via a Trotterised update scheme or the via the time-dependent variational principle. The success of this relies on the fact that low-lying excited states are also contained in area-law constrained corner. However, entanglement typically grows rapidly when a system is perturbed strongly requiring that χ increases to compensate. As a result time-evolving states can only be adaptively tracked for short timescales.

For 1D systems MPS form the basis of hugely successful methods such as density matrix renormalisation group (DMRG) [12, 1], that solves (A), and time-evolving block decimation (TEBD) [13], that solves (B). Both these algorithms scale as χ^3 and the reachable extreme of $\chi_{\text{MPS}} \sim 10,000$ has made a substantial amount of equilibrium physics and dynamics accessible for 1D systems governed by local Hamiltonians [1]. For 2D systems PEPS in principle can mimic the success of MPS in 1D once $\chi \sim 10$ owing to the higher connectivity of the network [2]. However, the higher order of the tensors in PEPS also presents a major computational barrier since algorithms to variationally optimise them scale as χ^{10} . Although polynomial this is nonetheless a formidable scaling that has severely limited practical calculations to an often insufficient $\chi_{\text{PEPS}} \sim 5$. The story is similar for MERA where the scaling for algorithms is χ^{13} [4]. So while MPS based algorithms have succeeded on workstations and turn-key commodity clusters with only mild optimisations, the further development of PEPS and MERA will necessitate the development of code tailored to high performance computing environments.

1.3 Aims of the TNT library

A common feature of the many varied tensor network geometries and algorithms is that they are built up

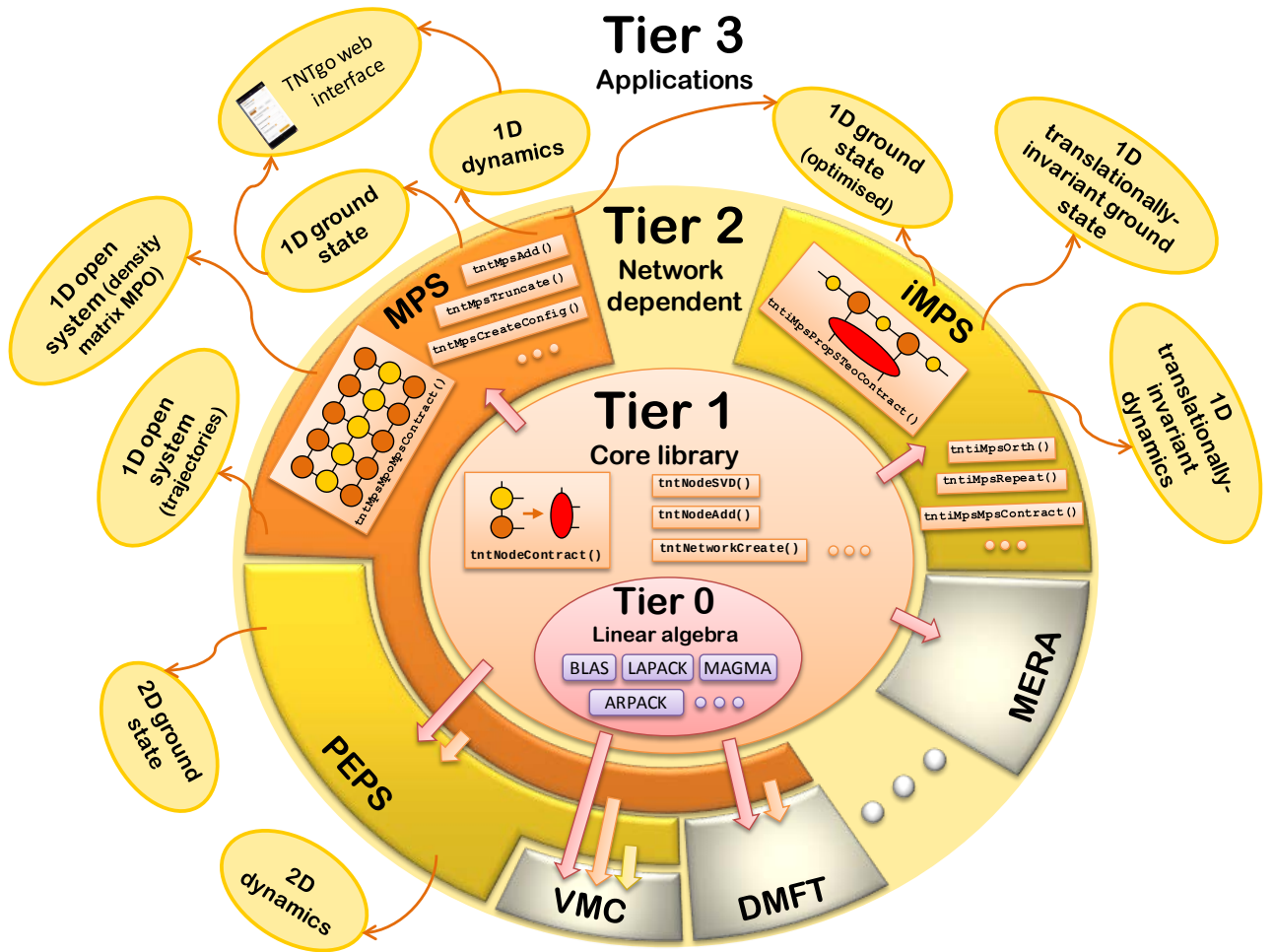


Figure 2: Structure of the TNT library. Tier 1 contains functions that do not depend on the network geometry. Tier 2 contains network-specific libraries, all of which are dependent on the Tier 1 core library. Additionally some Tier 2 libraries can be dependent on one another. The PEPS and iMPS (infinite MPS) libraries are still in development, and the VMC (Variational Monte Carlo) DMFT and MERA libraries are planned for future development. Tier 3 contains complete algorithms for performing simulations, which are comprised of Tier 2 ‘building blocks’.

from a few basic tensor operations. Essentially these boil down to *contracting* pairs of tensors to form new ones, *reshaping* tensors by combining or splitting legs, and correspondingly applying standard linear algebra operations to reshaped tensors that factorise them either via a singular value decomposition (SVD) or an eigenvalue decomposition. Thus on the face of it TNT algorithms are very simple and can easily be described using the graphical representation we employ here. However writing efficient software to perform algorithms quickly becomes complex. The numerous reshapes and re-ordering of the tensors, along with information keeping track of global physical symmetries on the indices [14] necessitates the need for efficient software to handle these manipulations.

Our aim in the TNT project is to provide a core library that from the outset provides completely gen-

eral but highly optimised routines for these ‘building blocks’. In particular it does not require users to deal with the details of the lower-lying tensor operations and takes advantage of the common tensor network picture. These building blocks are structured to make it straightforward to define new types of network geometries and the algorithms that act upon them. Routines for performing well-known algorithms in specific network geometries are also part of this project and are being built on top of the core library. There are several advantages to this approach:

- Enhancements in core tensor operations can be shared amongst all existing algorithms regardless of the type of tensor network being used.
- The source code for TNT algorithms is written

simply in terms of core tensor operations.

- New algorithms can be written with relative ease without any extra effort being required to use optimised tensor operations.

A key philosophy of the TNT project is to make it suitable for researchers with varied levels of expertise and interest in the TNT algorithms themselves. Therefore whilst the general core tensor routines simplify the design of new algorithms, it is also possible to access complete TNT applications which require only the parameters related to the physical system as an input.

Naturally these aims are also shared in part by other successful projects that provide high performance libraries based on tensor network methods, such as ALPS [15], POWDER [16], BlockDMRG [17], DMRGapplet [18], EvoMPS [19], DMRG++ [20], SnakeDMRG [21] and simpleDMRG [22], which focus on simulations of 1D quantum systems using DMRG and other well-known routines for MPS. Since the TNT project is based firmly on the tensor foundations it resembles more closely the philosophy of the iTensor [23] and Uni10 [24] projects, however there are differences in the approach taken here.

The major content of this paper focusses on the core TNT library of general tensor operations, and is structured as follows: in Sec. (2) we give an overview of the structure of the TNT project; in Sec. (3) we describe the objects that are used to define any general network or network node; in Sec. (4) we describe the most important routines that act on these objects that are independent of the system geometry; in Sec. (5) we discuss the performance of the library and finally in Sec. (6) we describe how to access the library and get involved in future development.

2 Structure of the library

The flexibility of the TNT library is achieved through the tiered structure shown in Fig. 2. Tiers 0 and 1 comprise the core library, on which all other tiers are based. It contains tensor routines that are completely general and do not depend on any aspects of the network or physical system. This part of the library is the focus of this paper. Tier 2 contains ‘plug-in’ libraries each of which relate to a specific network geometry, and contain routines for manipulating these networks. Tier 3 contains complete applications make use of one or more Tier 2 libraries to build an algorithm that accepts the physical parameters of the system as an input and outputs the required results. A complete description of Tier 2

and Tier 3, e.g. for MPS and PEPS algorithms, will be made in separate technical papers.

The source code for all tiers is written in C, which was chosen for performance and portability. These performance considerations are most important for the core tensor operations in tier 0, which contain the computationally heavy processing of the large arrays comprising the tensors in the network. Top level operations on networks are not as computationally intensive and so higher-level programming languages can be used without significantly impacting performance. For example Python can be easily interfaced with C and allows for a full object-orientated wrapper to be developed. This will be included in future releases of the TNT library.

We now give a brief discussion of each tier.

2.1 Tier 3: Applications

Tier 3 contains complete applications ready to be used for performing simulations e.g. for time evolution or for finding the ground state of a given system. Users of these applications are not required to have any knowledge of specific TNT algorithms, and the applications are described by the physical properties of the simulation rather than the algorithm underlying its operation. In some cases there may be more than one TNT algorithm suitable for performing a simulation: for example evolution of an open system using a Lindblad master equation can be performed either using quantum trajectories approach [25] or applying a full ‘super-operator’ to the density matrix of the system [26]. Here multiple TNT applications will be provided, with the choice governed by both the physical properties of the system and the computational resources and architecture available, to ensure that the most appropriate and efficient algorithm is used.

These applications can be interfaced with in multiple ways. For example, all simulation parameters can be given by passing command line parameters as options (e.g. `--Jzz 0.5` includes the term $\frac{1}{2} \sum_j \hat{S}_j^z \hat{S}_{j+1}^z$ in the Hamiltonian for a spin system and `--Ex2bdagb ap` calculates the expectation of $\hat{b}_i^\dagger \hat{b}_j$ for all pairs of sites i, j for a bosonic system). Alternatively, or in addition, parameters can be specified via an initialisation file. This allows more flexibility in the set up of the system e.g. allowing site dependent Hamiltonians to be defined or allowing two or more species in the system. In addition it also gives the flexibility for the final state saved from a previous calculation to be used as an input for any of the other initialisation file routines. Information about input and output formats is described in

Sec. (4.7). See Appendix A for a detailed example of using a command line Tier 3 routines.

2.2 Tier 2: Geometry dependent routines

Tier 2 contains routines that are specific to the network geometry. These routines are the building blocks of the TNT algorithms described in Sec. (2.1) – for example all the algorithms described use the routine for contracting a network formed of an MPS, its conjugate and a tensor network representation of an operator $\hat{O}$ illustrated in Fig. 3. This network represents $\langle \Psi | \hat{O} | \Psi \rangle$ for wave function $|\Psi\rangle$ and is used to find expectation values. All the routines for a specific network geometry are provided in a separate library. The library `libtntmps` is currently available, which contains a suite of routines which act on matrix product states with open boundary conditions. Future releases currently in development include `libtntipms` which will contain routines which act on infinite MPS systems and `libtntpeps` which will contain routines which act on the two-dimensional PEPS networks. All Tier 2 routines use Tier 1 functions to manipulate the nodes and networks, and thus are dependent on the core library. Additionally there may be dependencies between Tier 2 libraries e.g. the PEPS algorithms contains contraction steps that are MPS based.

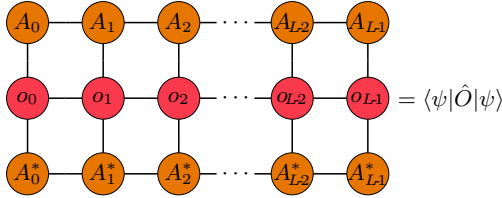


Figure 3: The tensor network the represents the expectation value of a matrix product operator

2.3 Tier 1: General routines

Tier 1 contains routines for manipulating the nodes in a tensor network. These routines are completely general and do not depend on network geometry. They include routines for modifying the tensor values through operations on the nodes, changing how the nodes are connected to one another in the network, and getting certain values (e.g. diagonal values) of the tensors. These routines are provided in single core library `libtnt`. The variable types and functions that make up this library are described in more detail in Sec. (3) and Sec. (4).

2.4 Tier 0: Linear algebra routines

Users of the library are not able to interface directly with this tier, however all tier 1 functions that modify tensor values are dependent upon it. These functions are concerned with the heavy-processing tasks of reshaping tensors to form matrices, which are then passed to external linear algebra libraries for optimised operations. The choice of linear algebra library linked to has a big impact on the performance of the algorithms, which is described further in Sec. (5).

3 Core TNT Variable types

The TNT core library describes networks of all forms using `tntNode` and `tntNetwork` variable types. The `tntNetwork` type provides a handle to the entire network, while the `tntNode` type defines all the nodes that make up the network. These are defined as so-called ‘opaque’ structures, so that its properties cannot be directly manipulated but are instead accessed through library functions. This ensures backwards compatibility of the code, allowing the flexibility to add or change details of the structures in later versions of the library. This also helps prepare for providing a fully object-oriented interface to the C-library in the future, where the variable types and their associated functions will map to objects and methods within a given class. Additionally there is a global `tntSystem` variable, which contains details of the simulation and system properties that are shared with all node and network operations.

3.1 tntNode

Every `tntNode` is associated with a tensor i.e. a multi-dimensional array of complex numbers. The `tntNode` also contains information about how it is connected in the network and symmetric properties of the node.

For the default `tntNode` type, the underlying tensor is a simple one-dimensional array of either real or complex numbers in memory, which represents the multidimensional array through information about the dimensions of and order of the indices. The tensors can also have additional structure as described below.

Each `tntNode` has one or more legs that map onto the indices of the underlying tensor. Each leg will initially map directly to a tensor index, although legs can be fused to create ‘fat’ legs that map onto more than one tensor index. Each leg of a `tntNode` can be connected to the leg of the

same or another `tntNode` to form a network. Legs that are connected represent tensor indices that will eventually be contracted and so only legs with the same dimension can be connected to each other. The legs are labelled and addressed by a single alphanumeric character.

A tensor can be associated with multiple nodes. When a node is copied the (usually large) tensor is simply linked to a new `tntNode` object, and so copying nodes is a cheap operation.

3.1.1 Symmetric nodes

Symmetric nodes are formed when there is a global physical symmetry in the system. In these cases conservation of the relevant quantum numbers requires that some elements of the tensor are always zero. By ordering the tensor indices suitably the non-zero elements of the tensor can be stored as a group of blocks, each block possessing a given quantum number.

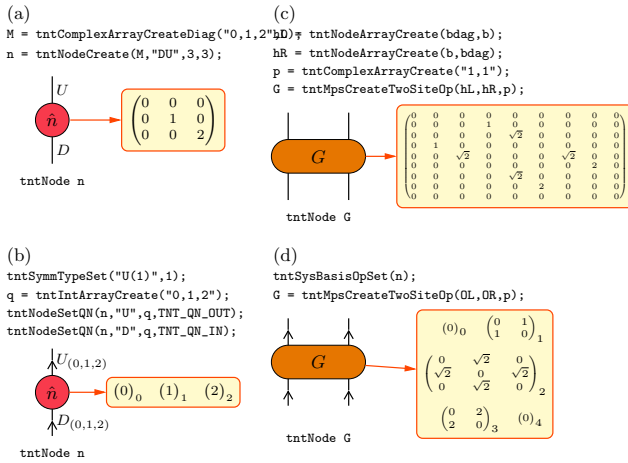


Figure 4: (a) The number operator for a bosonic system with no symmetry information. (b) After setting quantum numbers (which represent the possible number of bosons) on all legs the resulting non-zero blocks are simply 1×1 matrices for each allowed bosonic population on a single site up to a truncation (denoted by the subscript on the matrices). (c) Creating a two-site operator representing the hopping term $\hat{b}_j^\dagger \hat{b}_{j+1} + \hat{b}_j \hat{b}_{j+1}^\dagger$ without any quantum number information. (d) After the basis operator with symmetries has been set, creating a two-site operator results in a tensor having block-wise form. Here the quantum number of each block represents the number of bosons in the two-site basis of the incoming or outgoing physical legs.

The TNT library currently supports encoding of $U(1)$ symmetries (e.g. for particle number conser-

vation) on nodes. We follow the approach taken in Ref. [14], which achieves this by associating the relevant quantum numbers with each node. Briefly, each `tntNode` leg is marked as an incoming or outgoing leg, and every index on a `tntNode` leg corresponds to a quantum number label. Once all the quantum numbers on all the legs are set, only tensor elements for which the sum of the incoming quantum numbers equals the sum of the outgoing quantum number is retained, and is stored in a block labelled by that sum, as shown in Fig. 4. Making use of symmetries not only decreases the memory requirements but also substantially increases computation speed. This is because all tensor operations can be carried out for each quantum number sector, thus decomposing one large tensor operation into several much smaller tensor operations.

The quantum number labels are formed from one or more quantum numbers. For the simple case of conservation of particle number in a single-species system, each quantum number label is a single integer that corresponds to a given number of particles. In an m -species system each quantum number label is formed of m integers. This is illustrated in the code snippet below for a two-species systems, where the first species a can have 0 or 1 particles per site, and the second species b can have 0, 1 or 2 particles per site.

Listing 1: Setting symmetries in a two-species system

```
1 tntIntArray qn;
2 tntNode na;
3 tntComplexArray M;
4
5 tntSymmTypeSet("U(1)", 2);
6
7 qn = tntIntArrayCreate("0 0 0 1 1 1;"
8                        "0 1 2 0 1 2");
9
10 M = tntComplexArrayCreateDiag("0,0,0,1,1,1", 0);
11 na = tntNodeCreate(&M, "UD", 6, 6);
12
13 tntNodeSetQN(na, "U", qn, TNT_QN_IN);
14 tntNodeSetQN(nb, "D", qn, TNT_QN_OUT);
```

On line 5, symmetries are turned on, where the second argument specifies that there are two quantum numbers per label. Lines 7 and 8 create a two-dimensional array to hold the quantum number labels. The first row lists the quantum numbers for species a and the second row lists the quantum numbers for species b . The array thus defines the particle numbers in the single-site basis $\{0_a 0_b, 0_a 1_b, 0_a 2_b, 1_a 0_b, 1_a 1_b, 1_a 2_b\}$. Lines 10 and 11 create a node that corresponds to the number operator for species a , and the quantum numbers are assigned to it in lines 13 and 14.

Care must be taken to ensure that sufficient `tntNode` legs (and therefore tensor indices) are defined to allow an invariant tensor to be formed, otherwise elements will be discarded, and a warning output. As shown in Fig. 5 setting quantum number labels on the physical legs of an operator that results in a change of the total quantum number would cause all the elements to be discarded. However a covariant operator – one that changes a state from having a well-defined total quantum number label Q_1 to one having a different but still well-defined quantum number label Q_2 – can always be reformed as an invariant tensor by adding a singleton leg. Library functions are provided to determine such a suitable quantum number label for the additional leg.

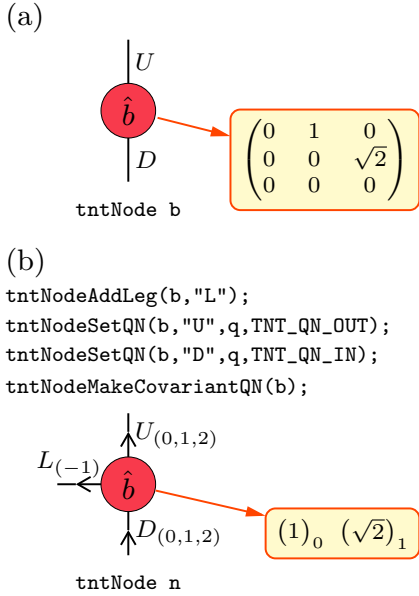


Figure 5: Changing a node to a symmetry preserving form. (a) Ladder operators such as $\hat{b}$ shown here change the total quantum number so are not symmetry invariant. (b) However they can be made invariant by adding a singleton leg carrying the appropriate quantum number, and this quantum number can be determined by calling a library function.

The framework we use to define $U(1)$ symmetries can be straightforwardly and easily extended to other Abelian symmetries e.g. Z_q within the library by updating the rules for adding quantum numbers on different indices in Tier 0. These have so far not been implemented due to a lack of user demand, but will be implemented if user requests (see Sec. (6)) are received.

In the future support for non-Abelian symmetries $SU(2)$ [27] will be added to the library. These symmetries are encoded in a similar way to Abelian

symmetries e.g. for the case of conservation of total spin the quantum number labels to assign would be composed of two numbers forming the spin index (j, m) . The difference between these two symmetry types is contained in Tier 0, where more complicated fusion rules are used when combining indices to reshape the tensors.

3.1.2 Functional nodes

A functional node is useful when the tensor elements represented by the `tntNode` depend on parameters that change often during a simulation, and are particularly useful when the parameters depend on values that are determined during the calculation itself. Rather than having static values, a functional node is defined by operators and a generating function that are fixed at the time it is created, and parameters that can be changed at any time using library functions.

Functional nodes can be loaded from an initialisation file (defined using a MATLAB library function – see Sec. (4.7) for more information about input and output). Alternatively, they can be created using core library functions as shown in Listing 2. First, the single site spin- $\frac{1}{2}$ operators are created from matrices in lines 6 to 12. Each operator is then contracted with itself to form a two-site operator (lines 14, 18 and 22), which are scaled by the time-step (lines 15, 19 and 23), and the matrices for these operators extracted and used to fill an array (lines 16, 20 and 24). The node is then created as a function of the two-site spin coupling terms on line 26. A functional node F can currently either have the form $F = \exp\{\sum_i p_i o_i\}$ or $F = \sum_i p_i o_i$, where p_i are the parameters, and o_i are the operators. Initially all the parameters are zero. The parameters are then set to form an operator representing propagation under the XYZ Hamiltonian in lines 29 to 31.

Listing 2: Creating and setting parameters for a functional node

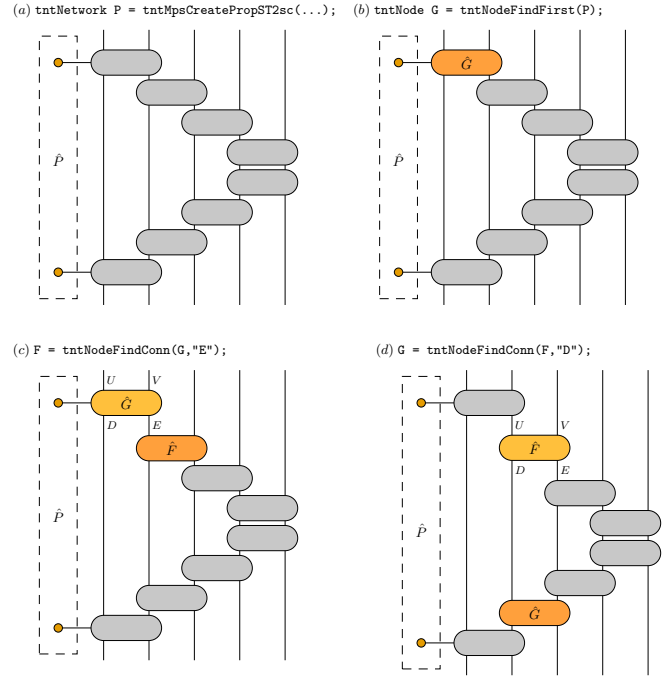
```
1  tntComplexArray Mx, My, Mz;  
2  tntNode Sx, Sy, Sz, 0, G;  
3  tntComplexArray Mo[3];  
4  tntComplex dt = {0, 0.01};  
5  
6  Mx = tntComplexArrayCreate("0 1; 1 0");  
7  My = tntComplexArrayCreate("0 -1i; 1i 0");  
8  Mz = tntComplexArrayCreateDiag("1 -1", 0);  
9  
10 Sx = tntNodeCreate(&Mx, "DU", 2, 2);  
11 Sy = tntNodeCreate(&My, "DU", 2, 2);  
12 Sz = tntNodeCreate(&Mz, "DU", 2, 2);  
13  
14 0 = tntNodeContract(Sx, Sx, NULL, "DU=EV");  
15 tntNodeScaleComplex(0, dt);  
16 Mo[0] = tntNodeGetMatrix(0, "DE", "UV");
```



```

17 0 = tntNodeContract(Sy, Sy, NULL, "DU=EV");
18 tntNodeScaleComplex(0,dt);
19 Mo[1] = tntNodeGetMatrix(0, "DE", "UV");
20
21
22 0 = tntNodeContract(Sz, Sz, NULL, "DU=EV");
23 tntNodeScaleComplex(0,dt);
24 Mo[2] = tntNodeGetMatrix(0, "DE", "UV");
25
26 G = tntNodeFuncCreate(Mo, 3, "exp", "DEUV", 2,
27                        2, 2, 2);
28
29 tntNodeSetRealParam(G,1.1,0);
30 tntNodeSetRealParam(G,1.2,1);
31 tntNodeSetRealParam(G,2.3,2);
32
33

```



3.2 tntNetwork

Networks provide a convenient handle to a number of nodes that are joined to one another. A network structure is formed of information about the start and end of the network. This is in the form of singleton terminating nodes that are connected to the first and last nodes in the network, represented as the small orange dots in Fig. 6. It does not contain a list of all the nodes in the network, but instead the network is defined as a linked list as shown in Fig. 6. This choice makes it very straightforward to insert, remove, contract, or factorise nodes in the network, since only adjacent nodes require updating. Furthermore many algorithms sweep through the network in a sequential pattern, for which a linked list is well suited. To get a handle to any node in the network, the first (or last) node in the network is found and the network then traversed using connections on the legs of the network.

A network can optionally also contain additional (geometry dependent) information. For example an MPS network can contain information about the coefficients for a Schmidt decomposition between each pair of sites in the network. Since this information is always obtained during MPS sweeping algorithms, keeping track of it does not add any extra computation cost.

3.3 tntSystem

The `tntSystem` is a global variable that contains a handle to the basis operator `tntNode`, which is used when generating any networks or nodes that

Figure 6: Example of creating and traversing a network - items in colour represent those that have a variable handle. (a) The network is created and the variable P contains information about the start and end points of the network. (b) Finding the first node in the network. (c) and (d) Finding other nodes in the network through connections to the currently selected node.

possess physical indices. The basis operator defines the physical basis for the system, for example in a bosonic system the basis operator would be the number operator $\hat{n}$. If there is a global physical symmetry it also contains the quantum numbers for each element of the physical index. Once the basis operator has been defined, any nodes created using library functions that are known to have physical legs can be generated in symmetric form automatically e.g. creating starting wave functions or network operators.

As well as this the `tntSystem` variable holds simulation parameters relating to the SVD type and truncation (see Sec. (4.1.2)), the system type (bosonic, fermionic, spin), and tunable parameters for the linear algebra routines (e.g. the maximum number of iterations in the sparse eigenvalue solver). These are first set to their default initial values, however they can be changed by calling core library functions (see Sec. (4.6)) or via command line functions. When this is done, a summary of the system information will be output to screen, a example of which is given below.

Listing 3: Printing out system information

```

1----- System Information -----
2No symmetry type set.
3No basis operator set.
4Relative truncation tolerance is 1e-16.
5Absolute truncation tolerance is -1.
6Truncation error tolerance is 1e-08.
7Tolerance for automatic blocking is -1.
8Current truncation type is 2norm.
9SVD type is LAPACK divide and conquer.
10Reshape re-use is turned on.
11Maximum number of iterations for eigenvalue solver
   is 300.

```

4 Core library functions

A full description of all core library functions is given in the documentation [28]. Here the most useful and commonly used functions are described.

4.1 Node algebra

All the functions which change values in the node are based on linear algebra routines, the majority of which call routines in the linked external libraries. These require the tensors to be reshaped into matrices or vectors first by re-ordering the indices appropriately, although this reshaping step is carried out in tier 0 and so it does not require user input. In some cases the user must specify which `tntNode` legs belong to the rows or columns of a matrix, and this is done simply by passing a string of the leg labels – the remaining manipulation is carried out automatically. This will be explained further by means of some examples below.

4.1.1 Contraction

Contracting two tensors together involves performing a sum over tensor indices. For example, consider a tensor with four indices A_{ijkl} and a tensor with three indices B_{kmn} : a rank-five tensor C can be formed by summing over the common index k

$$C_{ijlmn} = \sum_k A_{ijkl} B_{kmn}.$$

In practice, instead of performing a sum over tensor indices, the indices of A and B are reordered, such that the uncontracted indices appear first and last respectively. The contracted indices are assigned to the rows of A and the columns of B , and then the tensor contraction simply becomes equivalent to a matrix multiplication for which performance threaded linear libraries can be used.

A user of the library performing this contraction would connect `tntNode A` to `tntNode B` in a network with the legs that correspond to the index k – let us label these legs R and L respectively – then call the function to contract them. The reshaping is carried out automatically. Note that since leg labels are unique a *leg map* be required to relabel any remaining legs coming from A and B that have the same label. This is illustrated in Fig. 7.

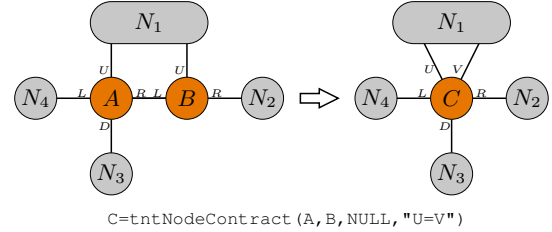


Figure 7: Contracting a pair of nodes within a network. Since both A and B have an uncontracted leg labelled U , the function call maps leg U of B to a new label V so that all the leg labels are unique on the new tensor C .

Many algorithms rely on contraction of a whole sequence of nodes, and it is important to do this in an order than minimises the computational cost [29]. This can be done using a function that performs contraction of a list of nodes. For three or four nodes the contraction cost is explicitly calculated for all permutations of contraction order, and the optimal order chosen. For more than four nodes, the nodes will be contracted in the order they are supplied, with only very minor optimisations. Namely: (a) any legs connected to legs on the same node (which is equivalent to a partial trace) are contracted first; (b) and connections of singleton dimension (tensor product) are always contracted last. An example of contracting multiple nodes for a common contraction that is part of the DMRG algorithm is shown in Fig. 8.

4.1.2 Singular value decomposition

An SVD factorises a matrix A into a product of three matrices $USV^\dagger$, where U and V are unitary matrices, and S is a real diagonal matrix containing singular values λ_i listed in decreasing order. To perform an SVD of a tensor having multiple indices, it must first be reshaped to a matrix by assigning the indices to either the row dimension or the column dimension. The row indices are then assigned to U and the column indices are assigned to $V^\dagger$.

A user performing an SVD of a node A into three new nodes U , S , and $V^\dagger$ would call `tntNodeSVD()`

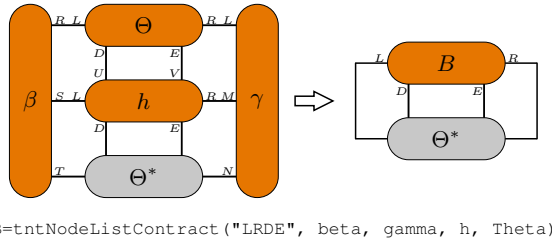


Figure 8: Contracting a group of nodes. The first argument of the function lists the new leg labels where the ordering of the legs is given by the order that the nodes that these legs originally belong to appear in the list. In this case the remaining legs after the contraction would be leg T of β , leg N of γ and legs D and E of h , which are relabelled as L , R , D and E on the resulting node B .

and simply list the labels for the legs of A that correspond to the rows as an argument. In addition the leg labels for the internal legs of U and V and both legs of S should be given. Optionally, the legs can be relabelled after performing the SVD by supplying a leg map. This is illustrated in Fig. 9.

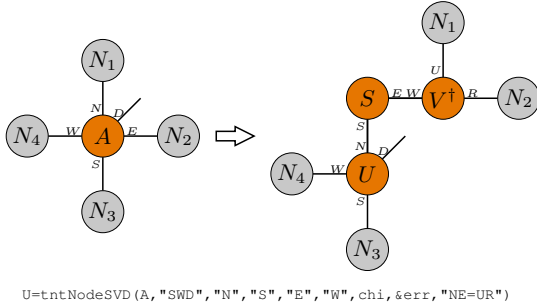


Figure 9: Performing an SVD on a multi-legged node connected to other nodes in a network.

If an exact SVD is performed, the dimension of the internal legs D_{exact} will be the minimum of the combined dimension of the legs assigned to U and the combined dimension of the legs assigned to $V^\dagger$. However in tensor network algorithms it is typical to truncate the internal dimension χ to a value less than this.

In all cases there will be an associated truncation error, which by default is calculated as

$$\epsilon_{\text{trunc}} = \sqrt{\left(\sum_{i > \chi_s} \lambda_i^2 \right)}.$$

The function used to calculate the truncation error can be changed easily if required (e.g. to the sum of the squares or the 1-norm of the discarded values).

Users of the library can perform a truncated SVD in four ways:

1. By passing a value of χ to `tntNodeSVD()` that is less than D_{exact} . This will discard all singular values $\lambda_i > \chi$.
2. By setting a global absolute truncation tolerance a . This will discard all singular values for which $\lambda_i < a$.
3. By setting a global relative truncation tolerance r . This will discard all singular values for which $\lambda_i / \lambda_0 < r$.
4. By setting a global truncation error tolerance ϵ_{tol} . This will discard the maximum number of singular values for which $\epsilon_{\text{trunc}} < \epsilon_{\text{tol}}$.

Note that when singular values in S are discarded, the associated singular vectors in U and V are also discarded. If two or more of the above bounds are used, then the one which results in the smallest internal dimension χ_s will be applied. The choice of the truncation error function and truncation error tolerances are stored in the global `tntSystem` variable.

4.1.3 Addition

Addition of nodes is carried out using the element-wise add function `tntNodeAdd()`, as shown in Fig. 10. Only nodes with an identical structure, i.e. the same number, labelling, and dimension of legs, can be added together using this function. If the symmetry information is identical, then addition will be carried out block by block.

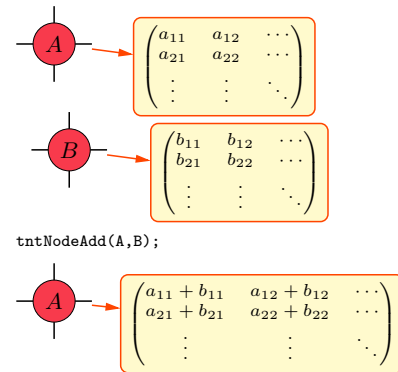


Figure 10: Adding two nodes together – calling the function adds each element of B to the respective element of A .

In some algorithms, a direct-sum or tensor-add is required, for example when adding two wave functions to one another by adding their MPS network

representations. In this case, the basis of the nodes must first be expanded, although this may not be on all legs/indices. This is performed using the function `tntNodeDirectSum()`. Such operations are crucial to certain TNT algorithms, for example strictly single-site DMRG [30] where a subspace expansion is performed on one internal leg as shown in Fig. 11(a). Another example is performing the addition of two wave functions $|\Psi_C\rangle = |\Psi_A\rangle + |\Psi_B\rangle$ in the MPS representation. To do this each node in the first MPS network is added to the corresponding node in the second MPS network, where the basis on both internal legs is expanded but the basis on the physical legs remains the same as illustrated in Fig. 11(b).

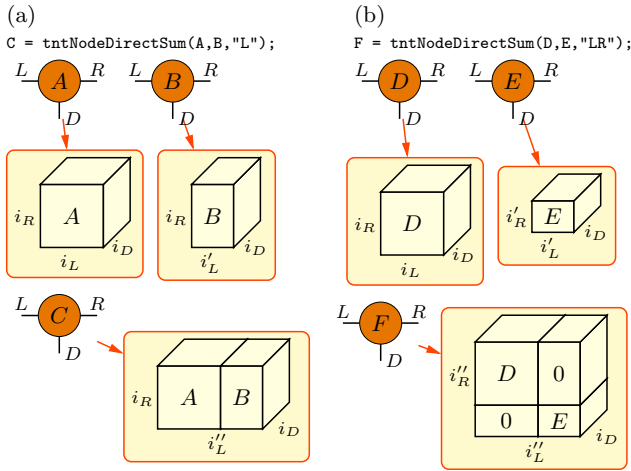


Figure 11: Performing a direct sum of two nodes. Here the node legs L , R and D are mapped to tensor indices i_L , i_R and i_D respectively. In (a) the direct sum expands the basis on leg L , and the new tensor has $\dim(i''_L) = \dim(i'_L) + \dim(i_L)$. The indices i_R and i_D should be identical. In (b) the direct sum expands the basis on legs L and R , so the new tensor has $\dim(i''_L) = \dim(i'_L) + \dim(i_L)$ and $\dim(i''_R) = \dim(i'_R) + \dim(i_R)$, with zeros inserted for elements of the tensor which correspond to the initial indices i_L, i'_R and i'_L, i_R .

4.2 Node utility routines

A number of utility routines are provided for performing basic operations on each `tntNode`. As described above, since the `tntNode` is an opaque structure a library function call is required to retrieve any information about it, or to perform any operations. Some of the most often-used routines include:

tntNodeCreate() Creates a new node, using either supplied tensor values or random tensor values.

See Listing 1 for an example of creating a node using this function.

tntNodeCopy() See Fig. 12. This does not copy the entire tensor but instead creates another node that points to the same data values. A conjugate copy can be taken, which simply adds a conjugate flag rather than conjugating all the values. If `tntNode` operations are later applied which change the tensor values, a new deep copy of the tensor is taken.

tntNodePrint*() A set of functions that provide different formats for printing out information about a `tntNode`, including printing out tensor values reshaped to a matrix (with the row legs and column legs specified in the arguments) or printing out the information (e.g. leg types, dimensions, symmetry properties) only.

tntNodeGet*() These functions retrieve values from the a `tntNode`, for example the first value, the diagonal values, or the trace of the tensor or the size of one of the legs (i.e. dimensions).

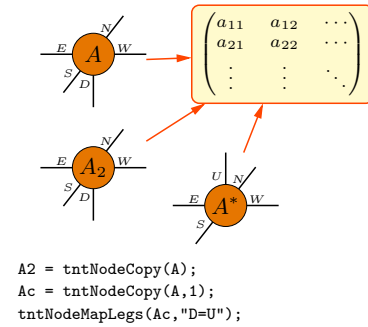


Figure 12: Copying an original node A to make an identical copy A_2 and a node that is the complex conjugate A^* . After the complex conjugate is taken the leg labels are mapped, so that A^* is labelled as an upwards-facing node rather than a downwards-facing node. None of these operations change the underlying values of the tensor or the ordering of the indices, so all nodes point to the same tensor. This makes a copying a node a cheap operation.

4.3 Changing node connections

There are also many functions which are not concerned with any of the tensor values, but only with the connections or properties of legs of the nodes. These include functions that allow any

general network to be constructed simply by joining `tntNode` legs together. In addition they include functions for modifying the properties of the `tntNode` legs. Some examples of these functions are listed below.

tntNodeJoin() Joins two nodes along the legs specified – see Fig. 13(a) and lines 17-19 of Listing 4. This means that any subsequent calls to contract which contains these nodes will result in a contraction along this index.

tntNodeInsert() Inserts a node between two nodes that are already connected. See Fig. 13(b).

tntNodeSplit() Removes all connections between a pair of nodes.

tntNodeSqueeze() This function removes the listed singleton legs from a node.

tntNodeAddLeg() This function adds singleton legs to a node.

tntNodeFuse() Fuses two legs together. Note this does not actually result in any change in the underlying tensor i.e. a reordering of indices, which could prove inefficient. See Fig. 13(d).

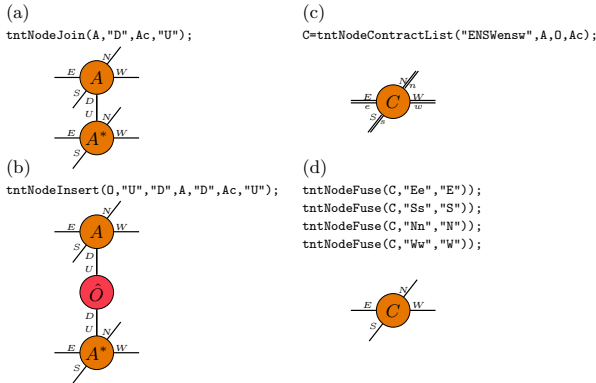


Figure 13: Illustration of some basic operations that can be carried out on nodes. (a) Two PEPS nodes are joined along their physical legs. (b) A single-site operator node is inserted between them. (c) The whole group of nodes are contracted leading to a node with 8 legs. (d) The legs are fused pairwise to result in a node with 4 legs.

4.4 Network algebra

There are some linear algebra routines that operate on entire networks rather than a single node or small group of nodes. These routines call external linear

algebra routines that are iterative and are designed to operate on matrices and vectors with a custom sparsity structure. Thus the external routines expect the result of matrix and vector operations, rather than the matrices and vectors themselves. Currently there is one linear algebra routine of this type in the core library, however more are planned in future releases.

Extremal eigenvalue solver

The function `tntNetworkMinSite()` is used to find extremal eigenvectors and corresponding eigenvalues of a 'matrix'. It calls an iterative routine (in Tier 0) that expects a 'matrix' times 'vector' operation to be calculated multiple times.

Note that `tntNetworkMinSite()` expects as an argument a network representing the matrix, a node representing the vector, and a pointer to another function (the contracting function) that performs the network contraction corresponding to the matrix-times-vector calculation. Taking a contracting function as an argument allows this function to be completely geometry independent.

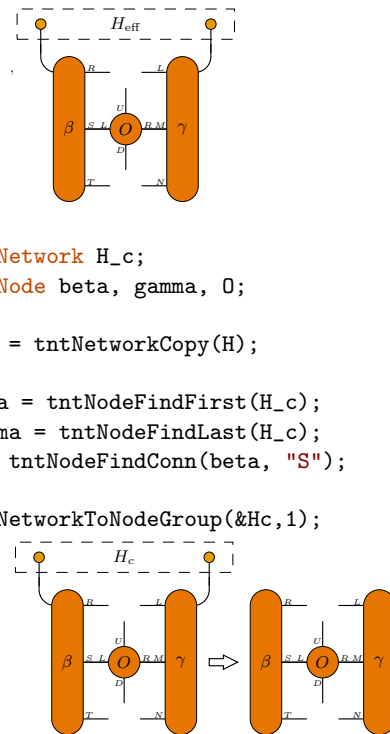
An example of a function that performs the network contraction is shown in Fig. 8. In this case the contracting function would insert the `tntNode` Θ in the network, perform the list contraction, and return the resulting node B .

Listing 4: Example of a function that defines a contraction sequence to be used as the matrix-vector multiplication function

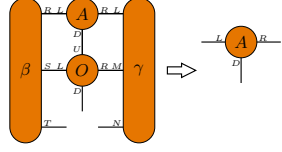
```

1 tntNode HeffA_contract(tntNode A,
2 tntNetwork H_eff)
3
4 {
5     tntNetwork H_c;
6     tntNode beta, gamma, 0;
7
8     H_c = tntNetworkCopy(H);
9
10    beta = tntNodeFindFirst(H_c);
11    gamma = tntNodeFindLast(H_c);
12    0 = tntNodeFindConn(beta, "S");
13
14    tntNetworkToNodeGroup(&Hc,1);

```



```

16
17   tntNodeJoin(A, "L", beta, "R");
18   tntNodeJoin(A, "R", gamma, "L");
19   tntNodeJoin(A, "D", 0, "U");
20   A = tntNodeListContract("LRD", beta, gamma, 0,
      A);
      
21
22   return A;
23 }

```

```

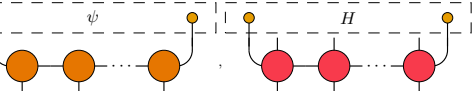
17
18   A_eig = tntNetworkMinSite(A, &H_eff, 1,
      &HeffA_contract, NULL, &E_eig);
19
20   tntNodeReplace(A, A_eig);
21
22   A = tntNodeFindConn(A, "R");
23 }
24
25   return E_eig;
26 }

```

For a more detailed example, see Listings 4 and 5. This shows code for a very simple, non-optimised one-site update to find the ground state MPS representation for an MPO Hamiltonian. The first function `HeffA_contract()` defines the matrix times vector multiplication, taking as arguments the node that defines the vector, and the network that defines the matrix. The second function sweeps through an MPS network left to right. For each site the network that defines the effective Hamiltonian is prepared (line 16) and the network minimisation routine `tntNetworkMinSite()` is then called (line 18).

Listing 5: *Performing variational minimisation on a group of nodes*

```

1 double ground_state_LR(tntNetwork psi,
2                       tntNetwork H)
3
4 {
5     tntNetwork psiHpsi, Heff;
6     tntNode A_eigv, A;
7     unsigned k, L;
8     double E_eig;
9
10    psiHpsi = tntMpsMpoMpsConnect(psi, H);
      
11
12    L = tntMpsLength(psi);
13    A = tntNodeFindFirst(psi);
      
14
15    for (k = 0; k < L; k++) {
16        H_eff = H_prepare(psiHpsi, k);

```

`tntNetworkMinSite()` takes as an input the initial guess for the eigenvector (the original MPS node A). This is used for the first call of `HeffA_contract()` – a pointer to which is also passed as an argument. The routine passes the resulting vector to a suitable iterative sparse eigenvalue solver. The solver returns a new vector, which is reshaped to a new node A to be passed to the contract function `HeffA_contract()` each time. Of course for different types of sweeping algorithms and networks, different network contract functions to `HeffA_contract()` could be devised.

4.5 Manipulating networks

There are some basic functions in the core library for manipulating networks, none of which depend on the network geometry. Due to this property they are limited in number, with the majority of network-level functions provided in the additional network-specific libraries.

tntNetworkCreate() Creates a new empty network. This network will contain no nodes – nodes can subsequently be inserted using `tntNodeInsertAtStart()` or `tntNodeInsertAtEnd()`.

tntNetworkCopy() Returns a handle to a network formed of copies of all the nodes in the original network, and all copies connected in the same way as in the original network. Like `tntNodeCopy()` these copies do not copy all the tensor values, but instead create additional pointers to the tensors. It is also possible to create a copy of a network with the complex conjugate of all nodes taken.

tntNetworkSplit() Splits a network into two separate networks. All the nodes involved in the split must be given.

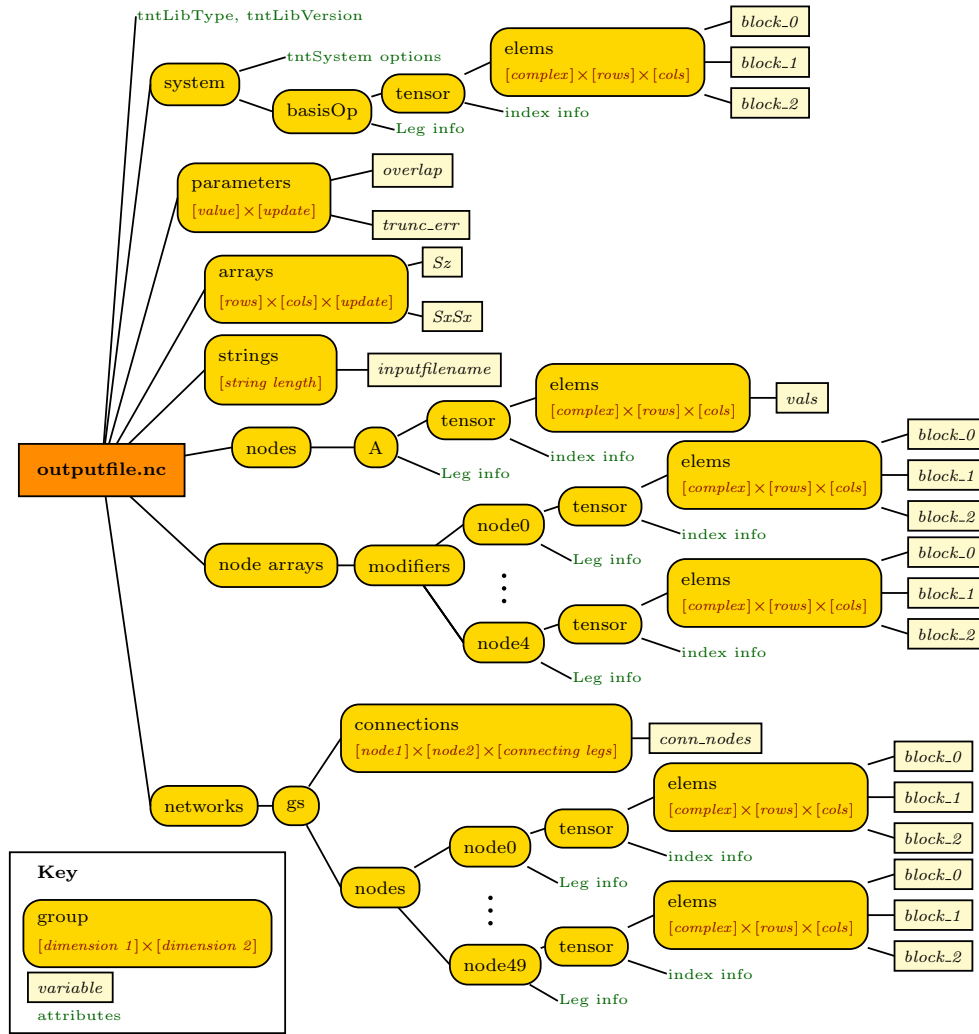


Figure 14: The hierarchical group structure for an example output file in NetCDF format. The groups each have several attributes, and contain the variables which hold the data in the form of arrays.

tntNetworkToNodeGroup() Deletes the network information (i.e. the network structure, and any network information) but leaves all connections between **tntNodes** that formed the network intact. This can be useful when joining two networks together to create a single network. It can also be useful when the entire network is contracted to a single node (for an example see line 14 of Listing 5) such that a network handle is no longer necessary.

4.6 System settings

There are a number of functions that can be used to change global calculation parameters, for example those used when performing a truncated SVD. For the full list please see the documentation [28] – only those related to functions already described above are given here.

tntSysInfoPrint() Prints all the current system

parameters to the standard outputs. See Listing 3 for an example of the output.

tntSVDTruncTo1Set() During an SVD, all singular values λ_i less than this will be discarded.

tntSVDRe1TruncTo1Set() During an SVD, all scaled singular values λ_i/λ_0 less than this will be discarded.

tntSVDTruncErrTo1Set() During an SVD, the maximum number of singular values will be discarded for which the truncation error is still less than this bound.

tntSVDTruncType() Set the function used to calculate the truncation error for all SVDs.

tntSVDTo1Set() Set the tolerance for zeroing values during the SVD (see Sec. (5)).

4.7 Input and output

The library supports input and output of data in MATLAB and NetCDF format, and MATLAB scripts are provided to convert between the two formats. In both cases the input and output data is richly structured to reflect the complex data structures that can exist in the library. As described above these complex structures are necessary to keep track of labelling and ordering of legs and indices in the simplest case, or more complicated block structures for symmetric nodes, as well as connections in networks of any general geometry. In addition all output files will contain the `tntSystem` structure and information about the library version. When such an output file is later used as an input file, the `tntSystem` structure is automatically loaded, ensuring that the simulation proceeds with the same system parameters (e.g. truncation tolerances, symmetry information).

MATLAB provides a convenient format and is already widely used within the community. However MATLAB is not suitable for all applications and/or users since it is a commercial package. Furthermore some computing resources, such as the UK National supercomputer ARCHER, do not have MATLAB installed as standard.

NetCDF 4 has been chosen as an alternative data format since it is also widely used within the scientific community, is freely available [31], and is already installed on many computing facilities. Like HDF5 (another commonly used scientific data format) it supports hierarchical group structures and is self-describing. However the interface for NetCDF 4 is considerably simpler whilst still containing all the flexibility required for description of TNT data structures (see Fig. 14).

For MATLAB output, nodes and networks are represented by means of custom ‘structure’-type variables which are created by functions in a separate MATLAB TNT library. Example initialisation scripts using these functions are available with the Tier 3 applications described in Sec. (2.1).

5 Performance

The design of the Tier 1 `tntNode` and `tntNetwork` variables and structures is intended to make developing TNT algorithms as simple as possible. However this does not come at the expense of performance of these routines. In fact, providing an opaque structure for these variables not only simplifies using the TNT library, but allows optimisations to be carried out in Tier 0

without any additional steps being taken by users.

As already described in Sec. (3.1.1) providing symmetry information on legs of the nodes leads to an internal block structure for the tensor. This not only reduces memory requirements: it also means that all linear algebra operations can be carried out block-wise. This can lead to large speed-ups (depending on the physical parameters of the system) - see Tables 1 and 2 for examples.

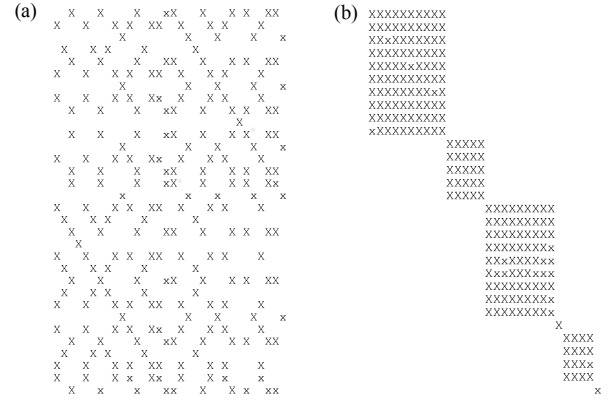


Figure 15: Transforming a dense array (a), with entries having magnitude greater than a given tolerance denoted by X , to a blocked matrix (b) by re-ordering the rows and columns.

However in some cases, the conserved quantities in the system can be difficult to determine, or cannot be assigned directly to the node legs. For these cases, it is possible to enable automatic blocking of the matrices. This comprises automatically transforming dense matrix blocks into a matrix having all non-zero entries in much smaller blocks on the diagonal. This is done by zeroing all entries smaller than a given tolerance, and reordering the rows and columns (see Sec. (4.6) for information on setting this). The matrices then passed to the SVD are much smaller, and furthermore can be processed in parallel. This is illustrated in Fig. 15 and described in more detail in an upcoming publication.

This leads to a vastly improved performance despite the overheads associated with determining the correct row and column order and performing this re-ordering. This is because the computational complexity of these linear algebra routines is $O(n^3)$ whereas the re-ordering operation scales as $O(n^2)$. Since the block structure is only realised when performing a linear algebra operation – the entire matrix is stored for all other operations – if the global symmetries of the system are known, it is far better to make use of them. However when they are not known turning on automatic blocking can still lead to significant speed improvements as shown in Table

Blocking type:	none	auto	U(1)
MKL/NAG/MAGMA ¹	617	549	161
MKL/NAG ²	555	552	135
MKL/ARPACK ³	2153	2231	377

Table 1: Performance of the TNT library for 1D ground state simulations. The times are given in seconds for a single DMRG sweep. All systems have 101 sites with physical dimension $d = 3$, internal dimension $\chi = 200$.

2.

Other optimisations carried out include optional re-use of reshape information when re-ordering indices. This minimises the computation time when tensors of identical dimensions are reshaped in the same way multiple times during a simulation, at the expense of some memory overhead. Determining whether this overhead is worthwhile depends on how many identical reshapes are carried out in a simulation, which will depend largely on whether the internal dimension is predominantly uniform throughout the system.

Care has also been taken to ensure that large amounts of data (which invariably belong to the tensor elements) are not copied or moved in memory unnecessarily, and to make use of shared-memory parallelisation where possible for the Tier 0 operations. Due to these efforts, we have tried to reduce as far as possible the time spent within the TNT routines themselves, with a large portion being spent in external linear algebra routines. When U(1) symmetries are not applied, we find that for large systems as much as 95% of the CPU time can be spent in linear algebra routines. When U(1) symmetries are used extra processing of the blocks increases the time spent in Tier 1 routines to around half of the total (but much reduced) CPU time. Therefore the choice of external library that is linked to when compiling the library is crucial for performance.

When choosing an external library, we ensure that we take advantage of the different computing architectures available to produce the best performance. As is now standard, we make use of shared-memory threading with the linked libraries. In addition, CPUs with an integrated GPU are becoming more widespread, and a version of the TNT library is available for these system types. On these systems we use the MAGMA linear algebra library and have found good performance when the matrix size is large enough (of order a few thousand). The core library routines will automatically determine whether to use GPU or CPU SVD based on the problem size. For a comparison of performance with

Blocking type:	none	auto	U(1)
MKL/NAG/MAGMA ¹	977	394	59
MKL/NAG ²	4565	516	57
MKL/ARPACK ³	5612	902	73

Table 2: Performance of the TNT library for 1D time evolution simulations. The times are given in seconds for a single TEBD time-step. All systems have 101 sites with physical dimension $d = 3$ and internal dimension $\chi = 1000$.

different library types see Tables 1 and 2. These calculations were carried out on a single node of the ARCUS-B cluster at the ARC [32] comprising an Intel E5-2640v3 Haswell 16 core processor with 64GiB and a NVidia K40 GPU. The simulation performed was the Tier 3 application `tntEvolve.if` with the initialisation file generated using the script `init_heis_s1.m` (available for download – see section 6 for further details).

In general, these times show the importance of choosing a high performance library, although it is worth noting a few points. Firstly, when symmetry information is being used, or χ is not sufficiently large, the size of the matrices sent to the linear algebra routines is much smaller, and thus using the highly parallelised GPU libraries is of limited benefit. Secondly, although the NAG library provides some optimised routines, the main benefit of this is for the sparse system solvers that are found in ARPACK. For algorithms that do not need to make use of these routines (such as TEBD), using a NAG enabled version does not have as great a benefit. Note that these calculations were also carried out using OpenBLAS⁴ but for the large system sizes used in these tests no results were obtained within a practical computation time.

6 Access and involvement

To use the library quickly and easily, we recommend using the ‘virtual box’ environment [33], shown in Fig. 16. Once the virtual machine is opened, no additional installation steps are required, and applications are pre-installed for a variety of systems. For step-by-step instructions on using the virtual

¹Linking Intel MKL 2015 11.2.2, NAG Fortran library FLL6i25DCL, CUDA 7.5.18, MAGMA 1.7.0 and compiled with Intel compiler 15.0.2

²Linking Intel MKL 2015 11.2.2, NAG Fortran library FLL6i25DCL and compiled with Intel compiler 15.0.2

³Linking Intel MKL 2015 11.2.2, ARPACK-NG 3.3.0 and compiled with Intel compiler 15.0.2

⁴Linking OpenBLAS 0.2.18, ARPACK-NG 3.3.0 and compiled with GNU compiler 4.9.2

machine, go to our CCPForge project page [34] and register to become a member of the project. The instructions can be found in the **Docs** section.

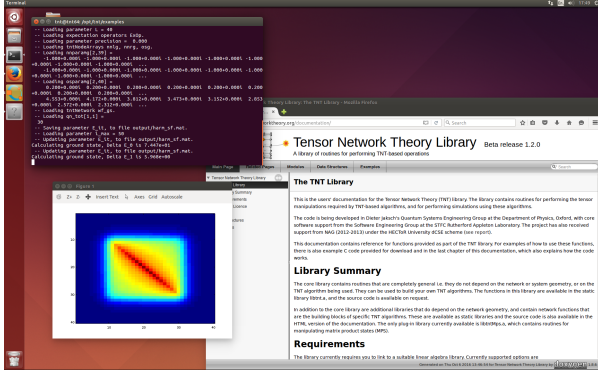


Figure 16: A screen shot example of running the TNT virtual machine, which has the TNT library pre-installed on an Ubuntu system. Here a calculation has just been run to find the single particle density matrix of the Bose-Hubbard model in a harmonic trap.

All the routines described in this paper are included in the library `libtnt`, which can be downloaded from the **Releases** section of CCPForge [34]. This library is required to run all TNT algorithms. Code for Tier 3 applications described in Sec. (2.1) and makefiles which can be modified to run on your own platform are also available for download in the **Releases** section. Getting the code running requires a C compiler (either gcc or icc), linear algebra libraries (LAPACK, BLAS and either ARPACK or NAG), and at least one of NetCDF or MATLAB libraries for input and output of data. The library `libtnt` can also be compiled directly from source code, which is the recommended approach for running on high-performance systems.

As well as providing a library for TNT simulations, one aim of the TNT project is to encourage community involvement. This is currently supported by the following features of CCPForge:

- A discussions forum, for posting information or questions about how to use the routines in the library.
- A feature requests form, where upcoming features can be viewed, and where anyone can request features they would like to see in future versions of the library.
- A bug reporting form. Once bugs are fixed you can choose to receive notification of this.

It is an eventual aim of the TNT project that users of the library will contribute to routines in Tiers

2 and 3, which will also be handled via CCPForge. Users who wish to contribute the TNT library can ask for write permission to a development branch of the SVN repository. Modifications can then be incorporated into the latest version of the library code. Each branch will be assigned a tracker item (similar to a feature request or branch report) where any notes or discussion of the feature, as well the differences made to the code, can be viewed. If the contribution meets the required standard (or can be modified easily by the TNT library developers to meet this standard), then the changes will be integrated in the main line of development code, and included in future official releases.

7 Summary and future developments

In this paper we have presented a high-performance yet easy to use library for performing tensor network simulations. The TNT library is completely general and does not depend on the network type. Here we have described the most important core tensor routines contained in tier 1 of the library (`libtnt`), and explained its overall structure.

In addition to this there is currently a tier 2 library (`libtntmps`) for performing matrix product state simulations in open boundary conditions. This will be described in detail in an upcoming technical paper similar in style to this one. Also tier 2 libraries for performing simulations of two-dimensional systems using PEPS (`libtntpeps`), and translationally invariant one dimensional systems (`libtntimps`) are under development. After their release additional technical papers will be provided to summarise their operation.

8 Acknowledgements

The authors would like to acknowledge the use of the University of Oxford Advanced Research Computing (ARC) facility [32] in carrying out this work. S.A. acknowledges support from the EPSRC Tensor Network Theory grant (EP/K038311/1).

References

- [1] U. Schollwöck. The density-matrix renormalization group in the age of matrix product states. *Annals of Physics*, 326(1):96–192, 2011.
- [2] F. Verstraete, V. Murg, and J. I. Cirac. Matrix product states, projected entangled pair states,

- and variational renormalization group methods for quantum spin systems. *Advances in Physics*, 57(2):143–224, 2008.
- [3] R. Orus. A practical introduction to tensor networks: Matrix product states and projected entangled pair states. *Annals of Physics*, 349:117 – 158, 2014.
 - [4] G. Evenbly and G. Vidal. Quantum Criticality with the Multi-scale Entanglement Renormalization Ansatz. *Chapter 4 in the book "Strongly Correlated Systems. Numerical Methods", edited by A. Avella and F. Mancini (Springer Series in Solid-State Sciences)*, 176, 2013.
 - [5] I. V. Oseledets. Tensor-train decomposition. *SIAM Journal on Scientific Computing*, 33(5):2295–2317, 2011.
 - [6] A. Cichocki. Era of Big Data Processing: A New Approach via Tensor Networks and Tensor Decompositions. *arXiv*, page 1403.2048, 2014.
 - [7] D. N. Basov, Richard D. Averitt, Dirk van der Marel, Martin Dressel, and Kristjan Haule. Electrodynamics of correlated electron materials. *Rev. Mod. Phys.*, 83:471–541, Jun 2011.
 - [8] I. Bloch, J. Dalibard, and W. Zwerger. Many-body physics with ultracold gases. *Reviews of Modern Physics*, 80(3):885–964, 2008.
 - [9] Daniele Nicoletti and Andrea Cavalleri. Non-linear light–matter interaction at terahertz frequencies. *Adv. Opt. Photon.*, 8(3):401–464, Sep 2016.
 - [10] Elbio Dagotto. Correlated electrons in high-temperature superconductors. *Rev. Mod. Phys.*, 66:763–840, Jul 1994.
 - [11] J. Eisert, M. Cramer, and M. B. Plenio. Colloquium: Area laws for the entanglement entropy. *Reviews of Modern Physics*, 82(1):277–306, 2010.
 - [12] S. R. White. Density matrix formulation for quantum renormalization groups. *Physical Review Letters*, 69(19):2863–2866, 1992.
 - [13] G. Vidal. Efficient classical simulation of slightly entangled quantum computations. *Physical review letters*, 91(14):147902, 2003.
 - [14] S. Singh, R. N. C. Pfeifer, and G. Vidal. Tensor network states and algorithms in the presence of a global U(1) symmetry. *Physical Review B*, 83(11):115125, March 2011.
 - [15] B. Bauer et al. The ALPS project release 2.0: open source software for strongly correlated systems. *Journal of Statistical Mechanics: Theory and Experiment*, 2011(05):P05001, 2011.
 - [16] G. De Chiara, M. Rizzi, D. Rossini, and S. Montangero. Density Matrix Renormalization Group for Dummies. *eprint arXiv:cond-mat/0603842*, <http://qti.sns.it/dmrg/> 2006.
 - [17] G. K.-L. Chan. Block Code for DMRG. <http://chemists.princeton.edu/chan/software/block-code-for-dmrg/>.
 - [18] T Köhler. DMRG-applet. <http://chemists.princeton.edu/chan/software/block-code-for-dmrg/>.
 - [19] A Milsted and T Osborne. evoMPS. <https://github.com/amilsted/evoMPS>.
 - [20] G. Alvarez. DMRG++ Website.
 - [21] C Guo, T Xiang, and J von Delft. snake-dmrg. <https://github.com/entron/snake-dmrg>.
 - [22] J. R. Garrison and R. V. Mishmash. Simple DMRG. <http://simple-dmrg.readthedocs.io/en/latest/index.html>.
 - [23] E. M. Stoudenmire and S. R. White. iTensor. <http://itensor.org>.
 - [24] Y.-J. Kao, P. Chen, Y.-H. Yun-Hsuan Chou, and C.-Y. Lai. Uni10, the Universal Tensor Network Library. <http://yingjerkao.github.io/uni10/>.
 - [25] A. J. Daley. Quantum trajectories and open many-body quantum systems. *Advances in Physics*, 63(2):77–149, 2014.
 - [26] Michael Zwolak and Guifré Vidal. Mixed-state dynamics in one-dimensional quantum lattice systems: A time-dependent superoperator renormalization algorithm. *Physical Review Letters*, 93(20), 2004.
 - [27] S. Singh and G. Vidal. Tensor network states and algorithms in the presence of a global SU(2) symmetry. *Physical Review B*, 86(1):195114, 2012.
 - [28] D. Jaksch S. Al-Assam, S.R. Clark. TNT library documentation. <http://www.tensornetworktheory.org/documentation>.

- [29] Robert N. C. Pfeifer, Jutho Haegeman, and Frank Verstraete. Faster identification of optimal contraction sequences for tensor networks. *Physical Review E*, 90(3):033315, sep 2014.
- [30] C Hubig, I P Mcculloch, U Schollwöck, and F A Wolf. Strictly single-site DMRG algorithm with subspace expansion. *Physical Review B*, 91, 2015.
- [31] Unidata NetCDF 4. *Boulder, CO: UCAR/Unidata Program Center*, page <http://www.unidata.ucar.edu/software/netcdf/>.
- [32] A Richards. University of Oxford Advanced Research Computing. August 2015.
- [33] Oracle Virtual Box. <http://www.virtualbox.org/>.
- [34] Maintained by Software Engineering Support Centre, STFC. CCPForge. <http://ccpforge.cse.rl.ac.uk/gf/project/tntlibrary>.